

COMPSCI 701:
Creating Maintainable Software
Lecture 3.1: Naming and Comprehensibility

Ewan Tempero, Yu-Cheng Tu
School of Computer Science

- PAQ

- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Potential Assessment Question

- Connect Four is game in which two players take turns dropping their pieces into a column of a 6x7 grid. A dropped piece falls as far as it can, either to the bottom or on top of the last piece dropped in that column. The first player that gets a line of 4 of their tokens wins.

You are given an implementation for Connect Four that includes a class called `Piece`. You play the game for 17 moves, which results in the second player winning. You note how many objects from the `Piece` class are created during this execution.

Which of the following statements is **most likely** incorrect regarding the game implementation?

- (a) 42 `Piece` objects are created — The `Piece` class represents a playing piece and all the `Piece` objects are created at the start of the game
 - (b) 17 `Piece` objects are created — The `Piece` class represents a playing piece and the `Piece` objects are created as each player takes their turn
 - (c) 6 `Piece` objects are created — The `Piece` class represents a playing piece and the `Piece` objects are only used for what is in a column in the grid.
 - (d) 1 `Piece` object is created — no idea what the `Piece` class represents, but probably not a single piece
- Bonus question: Suppose there are 9 `Piece` objects created. What's the most likely explanation for this?

Agenda

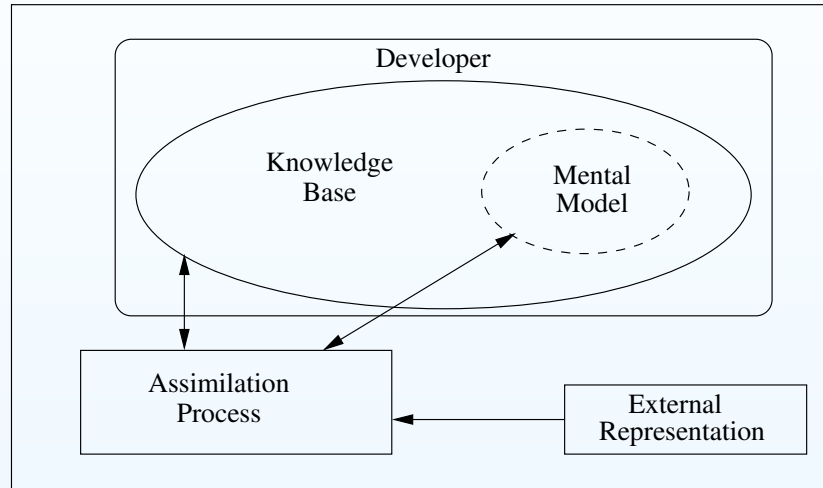
- PAQ
- **Agenda**
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- Admin
 - Assignment 1
 - Due: 8 August 23:59
 - Make sure to push your final implementation to your submission branch on GitHub
- Class exercise
- How names affect comprehensibility

- PAQ
- Agenda
- **Previously**
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Previously in COMPSCI701

- Michael P. O'Brien *Software Comprehension—A Review and Research Direction* TR UL-CSIS-03-3, University of Limeric, 2003.



- (Comprehensibility Definition) Degree of effectiveness and efficiency with which the implementation can be understood in order to conduct maintenance activities with confidence

- PAQ
- Agenda
- Previously
- **Class Exercise**
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Class Exercise

- Fibonacci sequence as described by Wikipedia

In mathematics, the Fibonacci sequence is a sequence in which each number is the sum of the two preceding ones. Numbers that are part of the Fibonacci sequence are known as Fibonacci numbers, commonly denoted F_n . The sequence commonly starts from 0 and 1...

- Given the following method. that returns the Fibonacci value at a particular index:

```
public int fib(int l) {  
    int[] f = new int[l+2];  
    f[0] = 0;  
    f[1] = 1;  
    for (int i = 2; i <= l; i++) {  
        f[i] = f[i-1] + f[i-2];  
    }  
    return f[l];  
}
```

- Answer and post the following questions (don't forget to post your team members)
 - Is `fib` a **good enough** name for this method? Why?
 - Is `fib` the **best** name for this method? If not, what's a better one?
 - Are the local variable names in this method **good enough**? Why?
 - Are the local variable names in this method the **best**? If not, what would be better?

Choosing Names for Comprehensibility

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- How can we use the comprehension model to decide which are the better names (for comprehension)?
- Assumptions:
 - putting facts we know (from KB) into MM is more efficient than putting new knowledge (from ER) into MM
 - accessing MM is more efficient than rest of KB
 - accessing KB is more efficient than accessing ER
- Names that help us better remember exactly what they are for are probably going to be better recorded in the MM, and generally require less reference to the KB or ER.
- People with really good memories probably create more complete MMs and can access them more efficiently even when the names are `method42`.

- PAQ
- Agenda
- Previously
- Class Exercise
- **Comprehension Model**
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Example: Using the Comprehension Model

- Noughts and Crosses as described by Wikipedia

Tic-tac-toe (American English), noughts and crosses (Commonwealth English), or Xs and Os (Irish English) is a paper-and-pencil game for two players who take turns marking the spaces in a three-by-three grid with X or O. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row is the winner. It is a solved game, with a forced draw assuming best play from both players.

- Consider choosing between the class names `Printer`, `Board`, and `Grid` for `NoughtsAndCrosses`
- Consider the comprehension tasks **find the class with the code that checks whether a move is legal (location is empty)**

- PAQ
- Agenda
- Previously
- Class Exercise
- **Comprehension Model**
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Example: Using the Comprehension Model

- Noughts and Crosses as described by Wikipedia

Tic-tac-toe (American English), noughts and crosses (Commonwealth English), or Xs and Os (Irish English) is a paper-and-pencil game for two players who take turns marking the spaces in a three-by-three grid with X or O. The player who succeeds in placing three of their marks in a horizontal, vertical, or diagonal row is the winner. It is a solved game, with a forced draw assuming best play from both players.

- Consider choosing between the class names `Printer`, `Board`, and `Grid` for `NoughtsAndCrosses`
- Consider the comprehension tasks **find the class with the code that checks whether a move is legal (location is empty)**
 - `Printer` — Look up KB and find nothing obviously related associating this name with checking legal moves.
 - `Board` — Look up KB and find nothing relating this name to Noughts and Crosses but do find general knowledge associating of moves to boards and so this might be the place to look
 - `Grid` — Look up KB and find the concept describes where moves are made, so very likely where checking a move is legal is done
- All depend on what's in the KB — maybe never heard of board games, maybe have already seen code that checks legal moves in a `Printer` class in another project
- All involve guesses, but with different levels of confidence
- All assume **how** the class is implemented (maybe the move checking is not done in `Grid` at all)
- But on balance, `Grid` will probably provide the best confidence

Determining Comprehensibility

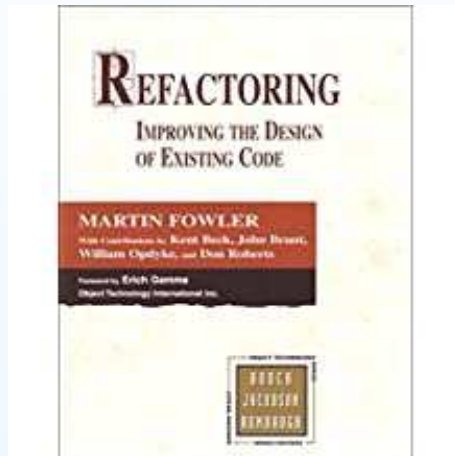
- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- **Advice**
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- With experience, we learn characteristics of code that probably make it harder or easier to understand
- Some of this experience has been “codified” in various ways, some of which we will look at
- Examples: Clean code, Code smells (both cover maintainability, not just comprehensibility)

Code Smells

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- **Code Smells**
- Clean Code
- Evaluating Martin
- Key Points

- Martin Fowler (feat. Kent Beck) *Refactoring: Improving the Design of Existing Code*. Addison-Wesley (2012)



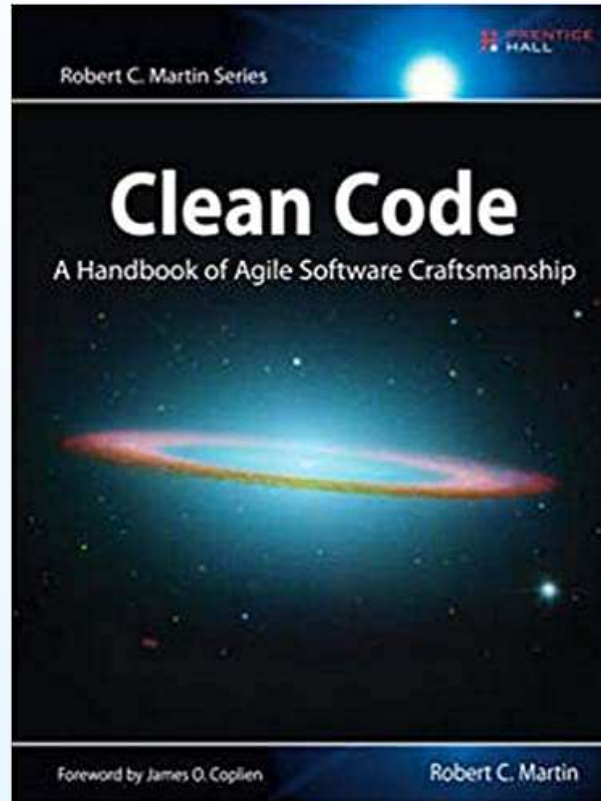
(There is now a second edition)

- Example smell: **Long Method**
 - “Since the early days of programming people have realized that the longer a procedure is, the more difficult it is to **understand**” (p77)
- Example smell: **Long Parameter List**
 - “long parameter lists are hard to **understand**” (p79)
- Example smell: **Large Class**
 - “a class with too much code is a prime breeding ground for duplicated code, chaos, and death.” (p78) *ok maybe not directly associated with comprehension but it's a fun quote!*
- Example smell: **Rename Method** (and other identifiers)
 - “Methods should be named in a way that communicates their intention” (p273) — presumably to aid **understanding**

Clean Code

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- **Clean Code**
- Evaluating Martin
- Key Points

- Robert C. Martin *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson (2008)



- code is read more than it is written $\Rightarrow$ write the code for the readers
- Much of the advice is about how to write code that can be understood

Chapter 2: Meaningful Names (Tim Ottinger)

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- Use Intention-Revealing Names
- Avoid Disinformation
- Make Meaningful Distinctions
- Use Pronounceable Names
- Use Searchable Names
- Avoid Encodings
- Avoid Mental Mapping (not what's in the comprehension model!)
- Class Names (should be nouns)
- Method Names (should be verbs)
- Don't be Cute
- Pick One Word per Concept
- Don't Pun
- Use Solution Domain Names
- Use Problem Domain Names
- Add Meaningful Context
- Don't add Gratuitous Context

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Use Intention-Revealing Names

- “The name of a variable, function, or class, should answer all the big questions. It should tell you why it exists, what it does, and how it is used. **If a name requires a comment, then the name does not reveal its intent.**” E.g. (from Martin)

```
int d; // elapsed time in days  
// or  
int elapsedTimeInDays;
```

- Evaluation:
 - Why are these questions we need to answer when comprehending code?
 - I see a use of a variable, in order to understand it then probably I need to answer at least one of these questions about it
 - I answer these questions by checking MM (most efficient), KB (next most efficient), or EP (least efficient)
 - e.g. the name `rowChoice` is likely to give me more information about what this variable does than the name `rc`
`player1RowChoice` even more so!

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- **Clean Code**
- Evaluating Martin
- Key Points

Avoid Encodings

- encodings = choosing names in a way that encode various information
- examples
 - a variable that is to hold an integer should start with **i**, **j**, or **k** (but not **l**!)
 - e.g. all field names should begin with **f**
 - e.g. all fields should be coloured a particular shade of **blue** (but which **shade**?)
 - Hungarian Notation (e.g. as used in Microsoft Windows C API), encodes what the type because there was essentially no type checking. e.g. (From Wikipedia)

arru8NumberList : variable is an **array** of **unsigned 8-bit** integers ("arru8");

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Hungarian Notation Comprehensibility

- Pros:
 - with no useful static type checking, when a name is encountered in the code, can determine what it is being used for — provided you can decode it (is it in the KB or do you need to use ER?)
 - if the compiler you use limits the length of identifiers, or you are not a very good typist and your “IDE” does not do auto-complete (comprehensibility?)
- Cons:
 - it is hard to remember the encoding (there are so many rules!), so usually end up going to ER

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- **Clean Code**
- Evaluating Martin
- Key Points

Other encodings (from Martin)

- **Member Prefixes** (e.g. `m_description`)
 - Martin: “Your classes and functions should be small enough that you don’t need them. And you should be using an editing environment that highlights or colorizes members to make the distinct.”
 - Pro: when dealing with a method accessing fields with parameters and local variables it can be more efficient to understand what’s going on if you can easily distinguish fields
 - Con: Martin: “prefixes become unseen clutter and a marker of older code” (comprehensibility?)
- **Interfaces and Implementations** (e.g. `IShapeFactory`)
 - Martin: “The preceding `I`, [...] and too much information at worst. I don’t want my users knowing that I’m handing them an interface.” (comprehensibility?)

Discussion: Evaluating Martin

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- Martin says we should use an editing environment that highlights members to make them distinct
 - Isn't highlighting a form of encoding?

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Discussion: Evaluating Martin

- Martin says we should use an editing environment that highlights members to make them distinct — Isn't highlighting a form of encoding?
- Martin says classes should be small enough that use of encoding is not needed — why would small classes remove the need?

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Discussion: Evaluating Martin

- Martin says we should use an editing environment that highlights members to make them distinct — Isn't highlighting a form of encoding?
- Martin says classes should be small enough that use of encoding is not needed — why would small classes remove the need?
- The code used in the class exercises using the convention (encoding) that all non-public fields begin with `_`. Martin says use of encodings clutters code — Is this true of `_`?

Discussion: Evaluating Martin

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

- Martin says we should use an editing environment that highlights members to make them distinct — Isn't highlighting a form of encoding?
 - Martin says classes should be small enough that use of encoding is not needed — why would small classes remove the need?
 - The code used in the class exercises using the convention (encoding) that all non-public fields begin with `_`. Martin says use of encodings clutters code — Is this true of `_`?
 - Martin says that use of encoding is a “marker of older code” — Why is this bad? How does code being old affect comprehensibility (or maintainability)?
 - Like any heuristic, not necessarily universally true
 - Begin field names with underscore (“_”). Provides a very visible way to distinguish fields from other identifiers (local variables, parameters), while creating lots of clutter
 - Name constants using all capital letters (Java code convention)
- ⇒ choose simple-to-interpret encodings

- PAQ
- Agenda
- Previously
- Class Exercise
- Comprehension Model
- Advice
- Code Smells
- Clean Code
- Evaluating Martin
- Key Points

Key Points

- Choose names to improve comprehensibility
- Much advice on how to choose names, mostly based on intuition and (different) people's experience rather than established science
- program comprehension model can explain (at least some of) the advice
- Choose names that are easy to add to the MM, or can be easily found in the KB. Avoid the need to go to the ER. (this isn't easy!)